

Projekt: Otoczka wypukła (Zespół nr 9)

Skład zespołu

- **Tomasz Czech** (Nr indeksu: 189803)
 - *Zadania:* Uzupełnienie dokumentacji projektu, przygotowanie instrukcji uruchomienia programu.
- **Mateusz Loska** (Nr indeksu: 189804)
 - *Zadania:* Analiza i wybór możliwych algorytmów, wstępne przygotowanie koncepcji skryptu, przygotowanie funkcji iloczynu wektorowego/testu orientacji.
- **Franciszek Spyra** (Nr indeksu: 189809)
 - *Zadania:* Implementacja algorytmów Grahama i Quickhull.
- **Mroczkowski Łukasz** (Nr indeksu: 189807)
 - *Zadania:* Optymalizacja i testy kodu źródłowego algorytmów. Współpraca nad dokumentacją.

Manual

Zadanie, które program ma realizować

Program wyznacza otoczkę wypukłą czterech punktów na płaszczyźnie:

- informuje, jakim zbiorem jest otoczka wypukła (czworokąt, trójkąt, odcinek, punkt),
- wypisuje współrzędne kolejnych wierzchołków otoczki wypukłej,
- wizualizuje wynik na wykresie przy użyciu biblioteki `matplotlib`.

Dane wejściowe: współrzędne czterech punktów na płaszczyźnie.

Lista opcji do wyboru z opisem

1. Wprowadzenie współrzędnych punktów

Program prosi o podanie dokładnie **4 punktów**. Każdy punkt należy wpisać jako dwie liczby oddzielone spacją.

- **Format:** `x y` (np. `2.5 3`)
- **Dozwolone wartości:** liczby rzeczywiste (`float`), dowolny zakres liczbowy.
- **Maksymalna dokładność zapisu punktu w programie:** 2 miejsc po przecinku.
- Jeśli użytkownik poda większą dokładność (np. `1.555`), wartość zostanie automatycznie zaokrąglona do 2 miejsc po przecinku metodą **half-up** (czyli `1.555` -> `1.56`).
- W przypadku błędnego formatu program zgłasza błąd i prosi o ponowne wprowadzenie danych.

2. Wybór algorytmu

Użytkownik wybiera jeden z trzech algorytmów wyznaczania otoczki wypukłej:

- `1` → Algorytm Grahama (*domyślny*)
- `2` → Algorytm Jarvisa
- `3` → Algorytm Quickhull

3. Tryb krok po kroku (debug)

- `t` → włącza szczegółowe logi działania algorytmu (każdy krok obliczeń wypisywany w konsoli)
- `n` → tryb normalny bez logów

4. Kontynuacja lub zakończenie

Po wyświetleniu wyniku i wizualizacji program pyta, czy wyznaczyć nową otoczkę:

- `t` → powrót do początku (nowe dane)
- `n` → zakończenie programu

Wymagania i instalacja

- Python 3.x
- Biblioteki zewnętrzne: `matplotlib`, `pytest`

Instalacja zależności:

```
pip install -r requirements.txt
```

Instrukcja uruchomienia

Krok 1 – Przejdź do folderu z projektem:

```
cd ścieżka_do_folderu_z_programem
```

Krok 2 – Zainstaluj zależności (tylko przy pierwszym uruchomieniu):

```
pip install -r requirements.txt
```

Krok 3 – Uruchom program:

```
python main.py
```

lub (w zależności od konfiguracji systemu):

```
py main.py
```

Przykładowe wywołanie (konsola):

```
--- PROGRAM DO WYZNACZANIA OTOCZKI WYPUKŁEJ 4 PUNKTÓW ---

Podaj współrzędne 4 punktów (x y):
Punkt 1: 0 1
Punkt 2: 1 0
Punkt 3: 2 0
Punkt 4: 0 4

Wybierz algorytm:
1 - Graham
2 - Jarvis
3 - Quickhull
Twój wybór (domyślnie 1): 1
Czy włączyć tryb krok po kroku (wypisywanie logów)? (t/n): n

=====
INFORMACJA: Otoczka wypukła jest zbiorem typu: **CZWOROKĄT**
-----

Współrzędne kolejnych wierzchołków otoczki wypukłej:
-> (1.0, 0.0)
-> (2.0, 0.0)
-> (0.0, 4.0)
-> (0.0, 1.0)

=====

Czy chcesz wyznaczyć nową otoczkę dla innych punktów? (t/n): n
Zamykanie programu. Do widzenia!

Process finished with exit code 0
```

Testy jednostkowe

Projekt zawiera katalog `tests/` z testami jednostkowymi napisanymi przy użyciu biblioteki `pytest`. Testy służą do weryfikacji poprawności obliczeń oraz jednolitości wyników między algorytmami.

Uruchomienie testów

```
pytest tests -v
```

Zakres testów

Klasa testów	Co jest sprawdzane
TestParsePoint	Walidacja danych wejściowych od użytkownika
TestCrossProduct	Poprawność funkcji iloczynu wektorowego
TestSquaredDistance	Poprawność funkcji kwadratu odległości
TestEdgeCases	Przypadki brzegowe i zdegenerowane dane
TestQuadrilateral*	Poprawność wyznaczania czworokąta (każdy algorytm)
TestTriangle	Poprawność wykrywania punktu wewnętrznego (trójkąt)
TestAlgorithmConsistency	Jednolitość wyników między algorytmami
TestCounterClockwiseOrder	Kolejność wierzchołków (przeciwna do zegara)

Testowane przypadki walidacji wejścia (TestParsePoint)

Program obsługuje następujące błędne dane wejściowe – każdy przypadek powoduje wyświetlenie komunikatu o błędzie i ponowne pytanie o dane:

- Litery zamiast liczb (np. abc def , a 3)
- Zły separator – przecinek (np. 1,2 lub 1, 2), średnik (1;2)
- Podwójna spacja lub tabulacja – **akceptowane** (traktowane jak jeden separator)
- Za mało liczb (np. 5) lub za dużo (np. 1 2 3)
- Pusty ciąg znaków lub same spacje
- Znaki specjalne (np. @ #)
- Minus bez liczby (np. - 3)

Testowana jednolitość wyników (TestAlgorithmConsistency)

15 zestawów punktów jest testowanych równolegle na wszystkich trzech algorytmach. Każdy zestaw sprawdza, czy Graham, Jarvis i Quickhull zwracają **identyczny zbiór wierzchołków** otoczki oraz czy liczba wierzchołków jest zgodna z oczekiwaną. Testowane przypadki obejmują m.in.:

- Kwadrat, romb, nieregularny czworokąt
- Trójkąt z punktem wewnętrznym
- Punkty współliniowe (wynik: odcinek)
- Współrzędne ujemne, mieszane, bardzo duże (do 10⁶)
- Bardzo płaski czworokąt (wysokość 0,01)
- Współrzędne całkowite i zmiennoprzecinkowe

Niezgodności z założeniami

Nie stwierdzono niezgodności. Projekt został zrealizowany zgodnie z założeniami zawartymi w treści zadania. Wprowadzono następujące rozszerzenia względem minimalnych wymagań:

- Zaimplementowano trzy algorytmy do wyboru (Graham, Jarvis, Quickhull) zamiast jednego.
- Dodano opcjonalny tryb debugowania wypisujący szczegółowy przebieg obliczeń.
- Dodano wizualizację graficzną otoczki przy użyciu matplotlib .
- Program umożliwia wielokrotne wyznaczanie otoczki bez konieczności ponownego uruchamiania.

Opis kodu

Lista plików z kodem źródłowym

```
Projekt/
├─ main.py                # Główna pętla programu - wczytywanie danych, sterowanie, wyświetlanie wyników
├─ algorithms/
│   ├── graham.py         # Implementacja algorytmu Grahama
│   ├── jarvis.py         # Implementacja algorytmu Jarvisa (Marsz Jarvisa)
│   └─ quickhull.py       # Implementacja algorytmu Quickhull
├─ utils/
│   ├── helpers.py        # Funkcje pomocnicze: iloczyn wektorowy, kwadrat odległości
│   └─ visualization.py   # Funkcja wizualizacji otoczki przy użyciu matplotlib
├─ tests/
│   ├── test_parse_point.py # Testy walidacji i parsowania danych wejściowych
│   ├── test_helpers.py     # Testy funkcji pomocniczych (iloczyn wektorowy, odległość)
│   ├── test_edge_cases.py  # Testy przypadków brzegowych i zdegenerowanych
│   ├── test_correctness.py # Testy poprawności wyników dla znanych konfiguracji punktów
│   ├── test_consistency.py # Testy spójności wyników między algorytmami
│   ├── test_order.py       # Test kolejności wierzchołków (przeciwnie do ruchu wskazówek)
│   └─ conftest.py         # Konfiguracja pytest i ustawienie ścieżek importu
├─ requirements.txt       # Lista zależności (matplotlib, pytest)
└─ readme.md             # Dokumentacja
```

Schemat algorytmu / Pseudokod

```

=== MAIN ===
1. Wczytaj 4 punkty (x, y) - z walidacją formatu
2. Wybierz algorytm (1 = Graham, 2 = Jarvis, 3 = Quickhull)
3. Wybierz tryb (debug t/n)
4. Wywołaj wybrany algorytm → otrzymaj listę wierzchołków otoczki
5. Określ typ otoczki wg liczby wierzchołków:
    1 → PUNKT | 2 → ODCINEK | 3 → TRÓJKĄT | 4 → CZWOROKĄT
6. Wypisz typ i współrzędne wierzchołków
7. Narysuj wykres (visualization.py)
8. Zapytaj o kontynuację

=== ALGORYTM GRAHAMA (graham.py) ===
1. Wyznacz p0 - punkt o najniższym Y (przy remisie: najniższe X)
2. Posortuj pozostałe punkty kątowno względem p0
    (bez trygonometrii - za pomocą iloczynu wektorowego)
3. Inicjalizuj stos: [p0, pierwszy_posortowany]
4. Dla każdego kolejnego punktu p:
    Dopóki stos >= 2 i skręt (stos[-2], stos[-1], p) <= 0:
        zdejmij stos[-1] (skręt w prawo lub wprost)
    Dodaj p na stos
5. Zwróć stos jako otoczkę

=== ALGORYTM JARVISA (jarvis.py) ===
1. Wyznacz punkt startowy - najbardziej lewy (min X)
2. Dopóki nie wrócimy do startu:
    Dodaj obecny punkt do otoczki
    Znajdź punkt "najbardziej na prawo" względem obecnego:
    dla każdego kandydata p:
        jeśli iloczyn_wektorowy(obecny, następny, p) > 0
        lub punkty są współliniowe i p jest dalej → zmień następny
    Przejdź do następnego
3. Zwróć otoczkę

=== ALGORYTM QUICKHULL (quickhull.py) ===
1. Wyznacz min_x i max_x (ekstrema poziome)
2. Podziel punkty na:
    lewy - po lewej stronie prostej min_x → max_x
    prawy - po lewej stronie prostej max_x → min_x
3. Rekurencja(zbiór, p1, p2):
    jeśli zbiór pusty → []
    znajdź najdalszy punkt od linii p1→p2
    podziel zbiór na dwa podzbiory (po obu stronach trójkąta)
    zwróć: rekurencja(lewy1) + [najdalszy] + rekurencja(lewy2)
4. Wynik: [min_x] + rekurencja(lewy) + [max_x] + rekurencja(prawy)

=== FUNKCJE POMOCNICZE (helpers.py) ===
iloczyn_wektorowy(p1, p2, p3):
    wynik = (p2.x - p1.x) * (p3.y - p1.y) - (p2.y - p1.y) * (p3.x - p1.x)
    > 0 → skręt w lewo | < 0 → skręt w prawo | = 0 → współliniowe

odleglosc_kwadrat(p1, p2):
    wynik = (p2.x - p1.x)2 + (p2.y - p1.y)2

```